

Interaction for the Digital Professor

Reference Notes and Prototype Concept

Jonathan Ma

September 13, 2026

1 Scope

This project aims to create a course-agnostic digital professor or teaching assistant. These notes examine its *interaction layer*: how students ask questions, share their work, and receive feedback. Statistics and engineering provide demanding initial examples because explanations frequently combine prose, equations, plots, and intermediate solution steps. Multimodal generative AI can connect these representations, but interface capability alone does not establish teaching competence or AGI.

Recommended initial experience: A chatbot-style web application with three tabs: Sources, Provider Settings, and Chat. On first use, the app prompts the student to upload a syllabus, extracts a proposed course name and basic course metadata, and asks the student to confirm or edit them. Students can then attach documents, images, equations, handwriting, or recorded speech and ask open-ended questions. One tutoring orchestrator identifies the student's intent and calls deterministic modality services. A visible *Office Hours* option communicates the longer-term direction, but synchronous office-hours tutoring is marked *Coming Soon* during the initial prototype.

Assume a small student team, ordinary laptops/tablets, internet access, modest paid API use, and no model training or dedicated GPU requirement. Exact prices, account availability, model identifiers, and preview status will be checked when implementation begins.

Part I

Existing and Emerging Interaction Methods

2 Comparison at a glance

Define implementation feasibility on three levels: high, medium, and lower. High indicates reuse of standard interfaces or hosted services, medium indicates we'll need to tune input integration, and low means we may need to build from the ground up. Effort is relative engineering effort, not a time estimate. Cloud services introduce usage charges and data transfer; locally runnable libraries introduce maintenance and hardware costs.

Dimension	Candidate technologies	Value and principal constraint	Initial feasibility
Text	Responses API; browser chat; KaTeX [1, 18]	Precise, searchable dialogue; generated explanations still need grounding and checking.	High; low effort. Core.
Voice conversation	OpenAI Realtime; Gemini Live; LiveKit Agents [2, 3, 4]	Natural follow-ups and interruptions; turn detection, reconnects, and shared visual context add work.	Medium; medium-high effort. Later.
Speech recognition / response	MediaRecorder; hosted transcription and TTS; faster-whisper; Web Speech API [5, 6, 7, 8, 9]	Spoken questions and read-aloud hints; symbols, accents, noise, and playback require testing.	High for push-to-talk; low-medium effort. Optional early.
Long-form audio	File transcription; timestamped segments; persistent session notes [6, 8]	Review an explanation or resume tutoring; chunking and summaries can lose details.	High for bounded recordings; medium effort. After core.
Documents	PDF.js; direct file input; Docling; hosted file search [10, 11, 13, 12]	Discuss notes and student work; reading order, scans, and citation accuracy vary.	High for selected pages; medium effort. Core.
Handwriting / stylus	Pointer Events + canvas; MyScript iinkTS; Mathpix [14, 15, 16]	Preserve derivations and sketches; ink capture is easier than reliable interpretation.	High for snapshots, medium for recognition. Early extension.
Mathematical expressions	MathLive; KaTeX; Mathpix; SymPy [17, 18, 16, 19]	Structured input, display, and bounded checking; notation and assumptions remain ambiguous.	High for entry/display; medium for checking. Core.
Image / camera	File upload; getUserMedia; vision model; Mathpix [20, 21, 16]	Share paper, plots, or diagrams; tiny labels and spatial reasoning can fail.	High for still images; low-medium effort. Core.
Tablet / iPad	Responsive web UI; Pointer Events; native PencilKit [14, 22]	Read, write, and discuss together; touch, keyboard, palm handling, and browser behavior need device testing.	High for web access; medium for polished pen UX. Core web.
Video	Gemini video understanding; Gemini Live; MediaRecorder [23, 3, 5]	Explain an evolving procedure; sampling, bandwidth, and temporal alignment complicate feedback.	Medium for clips; lower for continuous tutoring. Later.

3 Detailed technology notes

Each modality below is treated as a capability attached to the same chatbot conversation. The student asks a question in chat; the tutoring orchestrator identifies the intent, selects relevant sources and tools, and returns one coherent response.

3.1 Text-based interaction

- **Role in the chatbot:** The interaction default. Students ask open-ended questions, revise them, inspect prior feedback, and select actions such as *Hint*, *Explain*, *Check my work*, or *Quiz me*.
- **Technologies:** a hosted model through an API such as OpenAI Responses, streaming chat UI, conversation storage, and KaTeX for equation rendering.[1, 18]
- **Context handling:** send the current message, recent dialogue, selected course sources, and student artifacts to the tutoring orchestrator. Keep citations and source identifiers attached to the response.
- **Tutoring behavior:** ask for the student's attempt when appropriate, identify the likely intent, provide one instructional step at a time, and check whether the student follows before continuing. AutoTutor and newer tutor-evaluation work provide relevant dialogue precedents.[24, 29]
- **Feasibility: high.** Building chat is straightforward; pedagogical correctness, grounding, and useful comprehension checks require evaluation.

3.2 Voice: real-time and long-form conversation

- **Role in the chatbot:** a microphone control lets the student speak a message instead of typing. Future *Office Hours* can make the dialogue synchronous while keeping the same chat history and sources.
- **Approaches:** a cascade transcribes audio, sends text through the ordinary tutoring path, and optionally synthesizes the reply; native speech-to-speech uses a real-time multimodal session.
- **Technologies:** OpenAI Realtime or Gemini Live; LiveKit Agents can coordinate sessions.[2, 3, 4]
- **Initial flow:** use push-to-talk, show the transcript for correction, and then submit it as a normal chat message. This is especially valuable for terms such as "sigma squared" or "x sub i."
- **Long-form handling:** retain timestamped transcript segments, compact session summaries, original equations, and cited artifacts. File transcription or faster-whisper can process recordings.[6, 8]
- **Feasibility:** high for short recorded turns, medium for long sessions, and medium with greater effort for interruptible Office Hours. Turn detection, pauses, reconnects, and latency need testing.

3.3 Speech recognition and spoken responses

- **Role in the chatbot:** convert recorded questions into editable chat text and optionally read short tutor responses aloud while preserving the complete written response.
- **Technologies:** MediaRecorder for capture; hosted transcription models such as `gpt-4o-transcribe`; `gpt-4o-mini-tts` for speech output; faster-whisper for local inference; and the Web Speech API as a browser-dependent option.[5, 6, 7, 8, 9]
- **UX requirements:** provide transcript review, typing fallback, stop/replay controls, and spoken versions of notation rather than raw LaTeX. Identify synthesized speech as AI-generated.
- **Evaluation:** test mathematical vocabulary, signs, subscripts, negation, accents, background noise, and device speakers. Overall word error rate can hide a critical minus-sign or "not" error.
- **Learning consideration:** coordinate optional narration with the equation or diagram being discussed; multimedia-learning findings do not imply that every message should be spoken.[28]
- **Feasibility: high** as an optional chat input/output path.

3.4 Reading documents and student-provided content

- **Role in the chatbot:** the Sources tab stores the syllabus, course notes, and student work. Students enable a source, select a page or region, and refer to it naturally in chat.
- **Technologies:** PDF.js for viewing, direct model file input for bounded documents, hosted file search

for larger collections, Docling for controlled local conversion, and Mathpix for STEM OCR.[10, 11, 12, 13, 16]

- **Prototype flow:** begin by requesting a syllabus upload. Extract the likely course name, code, term, instructor, topics, and policies; show the proposed course name for confirmation rather than silently accepting it. Then accept pasted text and small PDFs, show processing status, preserve page images, and attach document/page/region IDs to the message. Responses link back to the supporting page.
- **Safeguards:** isolate course collections, distinguish authoritative material from a student's attempt, preserve tables and units, and treat instructions inside uploads as source content rather than application commands.
- **Feasibility:** **high** for bounded documents and **medium** for heterogeneous course ingestion. Test scans, columns, equations, figures, and tables separately.

3.5 Handwriting and stylus/pencil input

- **Role in the chatbot:** *Write* opens a canvas or expanded artifact view. The student selects a region and attaches it to a chat message such as "Check this step."
- **Technologies:** Pointer Events for cross-device stroke capture, MyScript iinkTS for browser handwriting recognition, and Mathpix for STEM image or stroke recognition.[14, 15, 16]
- **Processing stages:** keep capture, recognition, and interpretation separate. Preserve original strokes and order, create a snapshot, and show "I read this as ..." before reasoning over ambiguous notation.
- **Interaction design:** recognize on explicit submission rather than after every stroke. Prior studies support pen input's usability while showing that recognition errors and recovery remain central concerns.[25, 26, 27]
- **Feasibility:** **high** for capture and snapshots and **medium** for dependable recognition. Test fractions, subscripts, matrices, Greek letters, edits, and mixed diagrams.

3.6 Mathematical equations and expressions

- **Role in the chatbot:** equations may be typed directly, recovered from speech/OCR/ink, rendered inside messages, edited by the student, and attached to a request.
- **Technologies:** MathLive for structured entry, KaTeX for display, Mathpix for recognition, and SymPy for narrowly scoped symbolic checks.[17, 18, 16, 19]
- **Pipeline:** preserve the original input, display recognized LaTeX, confirm ambiguity, convert supported notation into a constrained representation, and run optional computation in a resource-limited service.
- **Limits:** equivalence checking does not verify assumptions, units, distributions, boundary conditions, or the reasoning connecting two steps. Permit "unable to verify." Never pass unsanitized strings to SymPy parsing that uses `eval`.[19]
- **Feasibility:** **high** for entry and display and **medium** for reliable checking.

3.7 Image and camera input

- **Role in the chatbot:** students attach a photograph, screenshot, plot, or engineering diagram from the Sources tab or message composer and ask about the full image or a selected region.
- **Technologies:** ordinary file upload, `getUserMedia` for an optional in-app camera, a vision-capable model, and specialized OCR when exact notation matters.[20, 21, 16]
- **Prototype flow:** upload, preview, crop or retake, select the relevant region, and preserve that selection with the chat turn.
- **Limits:** small labels, rotation, plots, and precise spatial relationships can cause errors. Request underlying data or a clearer crop when a value is critical.

- **Feasibility:** **high** for still images. Evaluate submitted work rather than faces or inferred emotional states.

3.8 Tablet and iPad interaction

- **Role in the chatbot:** the same responsive web app runs on laptop and tablet. Chat remains primary; sources and settings become drawers, and attached work can open in a larger writing view.
- **Technologies:** responsive web layout and Pointer Events initially; PencilKit is a later native option for drawing, stroke data, and emerging on-device recognition.[14, 22]
- **Device testing:** verify scrolling versus drawing, palm handling, pressure, orientation changes, virtual-keyboard obstruction, microphone/camera permissions, backgrounding, and audio playback on real devices.
- **Scope:** begin with one device per session. Paired laptop/tablet use adds session pairing and synchronization and should follow demonstrated need.
- **Feasibility:** **high** for responsive web access and **medium** for a polished native pen experience.

3.9 Video-based interaction

- **Role in the chatbot:** a later source type for uploaded demonstrations or recorded solution processes. Students select a timestamp or interval and ask a question about it.
- **Technologies:** MediaRecorder for capture, Gemini video understanding for bounded clips, and Gemini Live as a possible future real-time visual service.[5, 23, 3]
- **Limits:** frame sampling can miss brief actions or erased equations; live visual tutoring adds bandwidth, timing, interruption, and reference-resolution problems.
- **Prototype decision:** defer video. Use a paused frame or still image with a typed or spoken question. Office Hours should first reproduce professor-like dialogue and comprehension checks through text or voice.
- **Feasibility:** **medium** for bounded clips and lower priority for continuous video.

4 Websites and papers to explore

The links below form a practical reading list. Product and library pages support prototype implementation and capability checks; papers support interaction design, study design, and evaluation. The bibliography contains full references.

4.1 Implementation websites

Interaction area	Websites	What to examine
Text and multimodal models	OpenAI text generation and vision guide	Streaming text, image inputs, model limitations, and mixed-content conversations.
Voice and speech	OpenAI Realtime , Gemini Live , and LiveKit Agents	WebRTC/WebSocket sessions, interruptions, credentials, audio transport, and preview maturity.

Interaction area	Websites	What to examine
Transcription and spoken output	OpenAI transcription , OpenAI TTS , and faster-whisper	Hosted versus local recognition, timestamps, streaming, voice controls, and hardware needs.
Documents and retrieval	PDF.js , Docling , and OpenAI file search	Page rendering, layout extraction, OCR, chunking, retrieval, and source attribution.
Handwriting and tablets	MyScript iinkTS , Apple PencilKit , and Pointer Events	Stroke capture, recognition, pressure/tilt, browser coverage, network dependence, and on-device options.
STEM OCR and equations	Mathpix , MathLive , KaTeX , and SymPy parsing	Image/stroke OCR, editable math, rendering, symbolic checking, and safe parsing.
Camera and video	getUserMedia , MediaRecorder , and Gemini video understanding	Permissions, recording formats, frame sampling, temporal references, bandwidth, and privacy.

4.2 Relevant research papers

- **Conversational tutoring:** Graesser et al., *Intelligent Tutoring Systems with Conversational Dialogue*, introduces mixed-initiative tutoring dialogue and systems such as AutoTutor.[24]
- **Handwriting interaction design:** Anthony, Yang, and Koedinger, *A paradigm for handwriting-based intelligent tutors*, studies pen input and recovery from recognition errors.[25]
- **Handwriting and learning:** de Moraes and Jaques, *Does handwriting impact learning on math tutoring systems?*, compares typing and handwriting on desktops and tablets.[26]
- **Handwritten math tutor review:** Rodrigues et al., *Mathematics intelligent tutoring systems with handwritten input: A scoping review*, summarizes the research base and open access questions.[27]
- **Multimedia and spoken explanation:** Moreno and Mayer, *Verbal redundancy in multimedia learning*, informs decisions about narration, visible text, and graphics.[28]
- **Evaluating LLM tutors:** Maurya et al., *Unifying AI Tutor Evaluation*, proposes pedagogical dimensions for responses to student mistakes and confusion.[29]
- **Multimodal science tutoring:** Liu, Wang, and Zhang, *Is Your Multimodal Large Language Model a Good Science Tutor?*, argues that problem-solving accuracy alone does not establish tutoring quality.[30]
- **Step-level feedback:** Wang et al., *Stepwise Verification and Remediation of Student Reasoning Errors with Large Language Model Tutors*, studies verification of student steps before feedback.[31]

These papers should guide hypotheses and evaluation rather than be treated as proof that a particular API will improve learning in this project. Several examine narrow populations or subjects, and newer LLM studies may be preprints or benchmark studies rather than classroom trials.

Part II

Initial Prototype Sketch

5 Lightweight concept: a chatbot-style tutoring workspace

The prototype is an application or responsive web app organized around a persistent chat. Sources and provider settings occupy separate tabs, allowing the interface to remain focused on one task at a time. The system interprets whether the student wants an explanation, a hint, a step check, a summary, practice questions, or feedback on an uploaded artifact. This intent classification belongs to the tutoring orchestrator; the student does not need to select a rigid mode before every message.

The interface has three principal tabs:

1. **Sources:** on first use, this tab prompts for a syllabus. The system extracts a proposed course name and useful metadata, then asks the student to confirm or edit the course name. The tab subsequently holds course notes, PDFs, slides, images, camera captures, handwritten work, equations, and audio recordings. Each source shows its processing status and can be enabled or disabled for the current conversation.
2. **Provider Settings:** model/provider selection, optional transcription and speech providers, response style, course context, and advanced parameters. Sensible defaults allow students to ignore this tab; API secrets remain server-side rather than appearing in the client.
3. **Chat:** the default student-facing tab, containing messages, citations to attached sources, rendered equations, upload and recording controls, and tutoring actions such as *Hint*, *Explain*, *Check my work*, and *Quiz me*.

5.1 Syllabus-first onboarding

If no course exists, the app opens the Sources tab with the prompt *Upload your syllabus to create a course*. After ingestion, the document service returns structured candidates such as course title, course code, term, instructor, major topics, grading policies, and schedule. The UI prominently displays *We found: Introduction to Statistics* with *Confirm* and *Edit* actions. The confirmed name labels the workspace and chat history. Missing or ambiguous fields remain blank or marked for review; the system does not invent them. The syllabus remains the first authoritative course source and can be replaced or supplemented later.

5.2 Optional shared-work view inside the chat

When a student opens a document, image, or handwritten artifact, the chat may temporarily display a larger shared-work view. All input modes refer to the same selected artifact. This view supplements the tabbed chatbot interface rather than replacing it.

Example student journey.

1. Open the app and upload a syllabus when prompted in the Sources tab. Review the extracted course name and metadata, edit anything incorrect, and confirm creation of the course workspace.
2. Add notes, a homework PDF, and a photo or handwritten attempt in the Sources tab, then return to Chat.
3. Ask in chat, by text or recorded voice, “Why is this confidence interval step wrong?” The system

Digital Professor Course: Statistics		Office Hours (Coming Soon)
Sources	Provider Settings	Chat
Course created from syllabus: Introduction to Statistics [Review course details]		
Student: Why is my confidence interval step wrong?		
Digital Professor: I read your selected expression as $\bar{x} \pm 1.96 \frac{s}{\sqrt{n}}$. Is the population standard deviation known?		
Source: Lecture notes, page 4		
[Hint] [Explain] [Check my work] [Quiz me]		
Ask anything... [Attach] [Record] [Send]		

Figure 1: Proposed tabbed interface with Chat active. Sources handles syllabus onboarding and later uploads; Provider Settings contains model and speech choices. *Office Hours* remains marked *Coming Soon*.

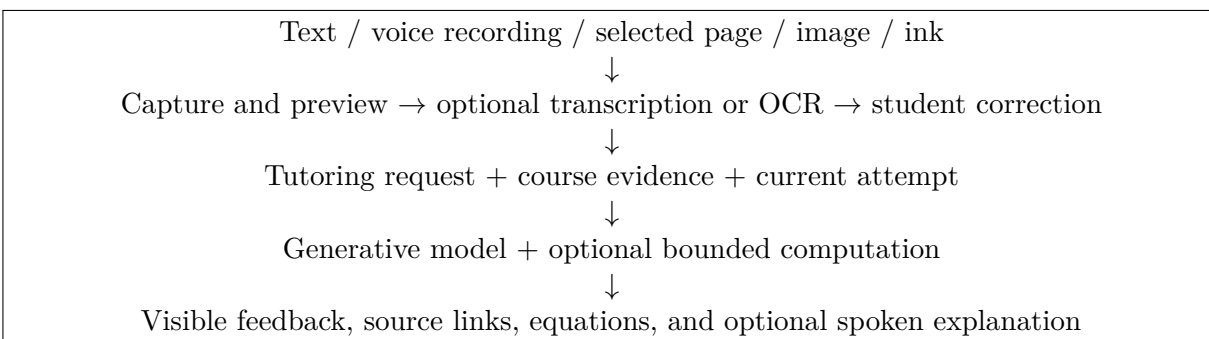
Digital Professor Course: Statistics Problem: Confidence interval	
Shared worksheet Lecture notes, page 4 [Choose page] Student's selected line: $\bar{x} \pm 1.96 \frac{s}{\sqrt{n}}$ [Photo / PDF / Write / Type equation] Recognized expression appears here. [Correct notation] [Use this expression]	Tutor conversation Student: Why is this step wrong? Tutor: Is the population standard deviation known, or are you estimating it? [Hint] [Explain] [Quiz me] Source: lecture notes, page 4 [Read aloud] [Stop audio]
[Select region] [Check this step] Type a question... [Record question] [Review transcript] [Send]	

Figure 2: Proposed tablet landscape or desktop view. On narrow screens, switch between worksheet and conversation while preserving the selection. Example dialogue is illustrative, not output from an evaluated system.

- infers that the student wants step-level feedback and determines which attached material is relevant.
4. The orchestrator calls document retrieval, image/OCR, transcription, or equation handling only when required. Ambiguous transcription or notation is shown to the student for correction.
 5. For a small independent normal sample with unknown population standard deviation, discuss the appropriate Student's t interval and its assumptions. The tutor links to the supplied course explanation rather than inventing a page citation.
 6. The student revises the expression to $\bar{x} \pm t_{1-\alpha/2, n-1} s / \sqrt{n}$, explains the choice, and requests feedback. The tutor gives a follow-up question and saves a short progress note.

The same interaction can discuss a free-body diagram or a written argument by changing course materials and evaluation criteria. This is the operational meaning of course-agnostic: reusable interaction and context handling, with domain-dependent correctness checks.

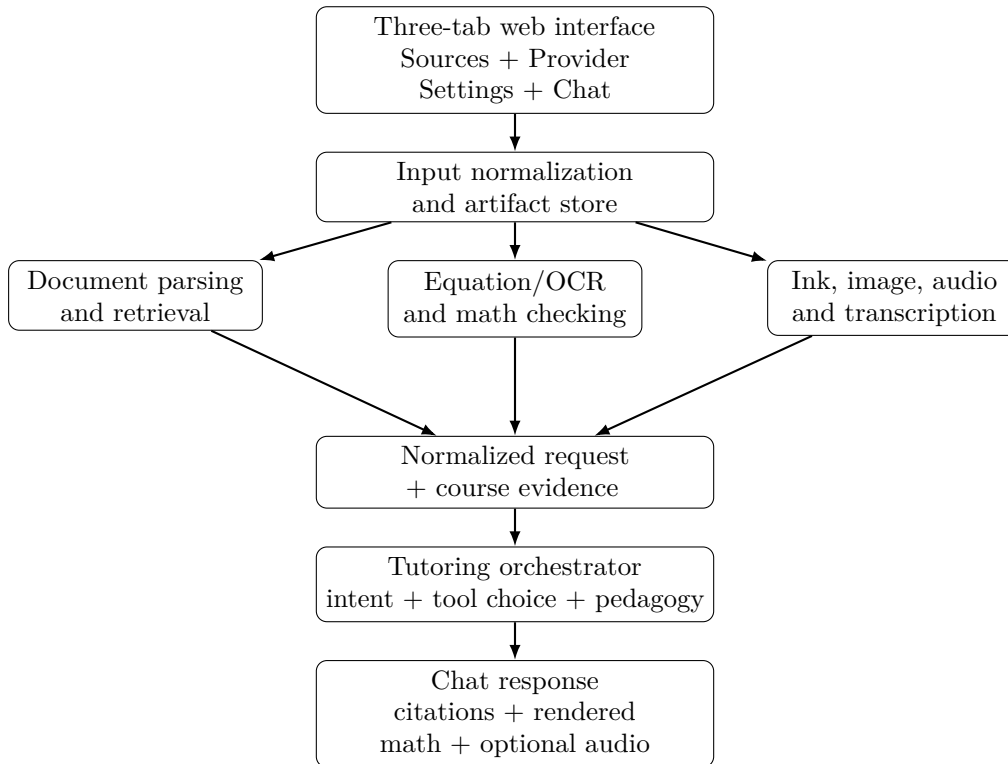
Minimal conceptual flow:



Keep these boundaries replaceable. A useful request record contains a session ID, course ID, question, inferred intent, attachment/page/region references, original input, corrected notation, active providers, and requested response format. Provider-specific adapters can change without rewriting the student's workflow.

6 Initial architecture: one orchestrator with modality tools

The first prototype should use a single tutoring orchestrator rather than a manager delegating to multiple autonomous agents. Document ingestion, equation rendering, tablet capture, OCR, transcription, and speech synthesis are mostly bounded processing tasks. Implement them as services or tools that return structured results. This retains modularity while avoiding extra model calls, prompts, latency, cost, and coordination failures.



A modality service should return structured data rather than a conversational answer. A result can contain the input modality, original artifact ID, recognized text or LaTeX, confidence, provenance, and a `needs_confirmation` flag. The tutoring orchestrator receives these results with course evidence, determines the student’s intent, selects any further tools, and produces one coherent student-facing response.

7 Implementation scaffold

The first implementation should run locally in a browser and use mock data before connecting to a model or document-processing backend. This makes the interaction design demonstrable while the parallel research on syllabus extraction, model selection, costs, and existing products continues. The mock responses and service contracts can later be replaced without changing the tabs or student workflow.

7.1 Recommended project structure

```

research/interaction-methods/
  webapp/
    frontend/
      package.json
      src/
        app/
          App.tsx
          routes.ts
        components/
          TabNavigation.tsx

```

```

    SourceCard.tsx
    ChatMessage.tsx
    MessageComposer.tsx
    EquationPreview.tsx
  tabs/
    SourcesTab.tsx
    ProviderSettingsTab.tsx
    ChatTab.tsx
  services/
    apiClient.ts
    mockApi.ts
  types/
    course.ts
    source.ts
    chat.ts
    provider.ts
  fixtures/
    extractedCourse.json
    conversation.json
  backend/
    app.py
  src/
    digital_professor/
      config.py
      ingestion.py
      recognition.py
      schemas.py
      skills.json
  notebooks/
    recognition_demo.ipynb
  resources/
    interaction_test/

```

The existing files in `resources/interaction_test/` provide the first coherent test course: a Nonlinear Optimization I syllabus, lecture notes, homework, correct handwritten work, incorrect work, and ambiguous handwriting. The repository should later add audio and image-format fixtures, but these PDFs are sufficient to scaffold upload states and the main source-grounded chat path.

7.2 Frontend architecture

- **Stack:** React with TypeScript and Vite for a lightweight single-page application served on localhost. React supplies reusable stateful components, while Vite supplies the local development server and production build.[\[41, 42\]](#)
- **Navigation:** a top-level tab state switches between Sources, Provider Settings, and Chat without discarding the active course or conversation. Chat is the default tab after onboarding.
- **Sources tab:** if no course exists, show *Upload your syllabus to create a course*. While using mocks, uploading `samp_syllabus.pdf` loads a fixture containing the proposed name *Nonlinear Optimization I*, course code `EN.553.761`, term, and topics. The student must confirm or edit the name. After confirmation, the tab accepts additional files and shows *queued*, *processing*, *ready*, or *failed* states.

- **Provider Settings tab:** begin with *Mock provider* and placeholders for the tutor, transcription, OCR, and speech providers. Store only provider IDs and user preferences in browser state. API keys remain backend environment variables.
- **Chat tab:** render user and tutor messages, citations, math blocks, attachments, and comprehension checks. The composer accepts text and file references. Microphone and equation controls may initially display *Coming Soon* or return mock results.
- **Office Hours:** keep a visible disabled control labeled *Coming Soon*. It should not require any real-time implementation in the scaffold.
- **State:** for the first demonstration, keep the active course, source list, settings, and messages in a single application state object. Add browser persistence only after the basic reset and upload flows work predictably.

7.3 Frontend components and states

The UI should be built around states that can be demonstrated without a backend:

- **Empty state:** no course exists; the Sources tab requests a syllabus.
- **Extracting state:** show the uploaded filename and a processing indicator.
- **Confirmation state:** display extracted fields with editable course name and *Confirm course* action.
- **Ready state:** display the course name in the application header and allow notes, homework, and student work to be added.
- **Chatting state:** accept an open-ended question, show which sources are active, and return a fixture response with a citation and comprehension check.
- **Recovery state:** if recognition confidence is low, show the interpreted text or equation and require correction before continuing.

One scripted mock response should use the uploaded optimization materials. For example, after the student attaches `samp_solution.pdf` and asks whether the log-sum-exp proof is complete, the tutor can ask the student to justify why $\text{diag}(p) - pp^\top$ is positive semidefinite, cite the relevant lecture-note page, and finish with a comprehension question.

7.4 Backend architecture for the next stage

After the frontend scaffold is stable, add a small Python backend using FastAPI. FastAPI provides typed request models and automatically generated interactive API documentation, which is useful while the contracts are still changing.^[43] The backend remains a thin boundary around reusable Python functions:

- POST `/courses/extract`: accept a syllabus and return proposed course metadata.
- POST `/courses`: confirm the edited metadata and create an in-memory course record.
- POST `/sources`: register a document or student artifact and return its processing state.
- POST `/chat`: accept the normalized tutoring request and return a structured tutor response.
- GET `/providers`: return the provider registry supplied by the model-and-cost research.
- GET `/health`: verify that the local backend is available.

For the first backend demonstration, use in-memory dictionaries and fixture responses. File persistence, a vector database, authentication, background workers, and production deployment are outside this scaffold. When a real model is added, only the backend reads credentials and calls provider APIs. The browser sends provider IDs, never secret keys.

7.5 Shared data contracts

The frontend, backend, and notebook should use the same conceptual objects:

- **CourseMetadata:** course ID, confirmed name, extracted name, code, term, instructor, topics, confidence, and fields needing confirmation.
- **SourceArtifact:** artifact ID, course ID, filename, media type, role (authoritative source or student work), status, page count, and optional selected region.
- **TutorRequest:** session ID, course ID, message, inferred intent, active source IDs, attachment IDs, recognized text/math, and response preferences.
- **TutorResponse:** response type, message, rendered-math strings, citations, recognition confirmation, comprehension check, and confidence.
- **ProviderDefinition:** provider ID, model ID, modalities, streaming support, enabled state, and cost fields supplied by the parallel research.

Use JSON fixtures conforming to these contracts before introducing network calls. This gives the syllabus, provider, frontend, and backend work a shared integration target.

7.6 Local setup and development sequence

The intended local workflow is:

1. Create the `research/interaction-methods/webapp/frontend` application with the React–TypeScript Vite template, install dependencies, and run the Vite development server. Vite normally serves the application at <http://localhost:5173>.^[42]
2. Implement the tab layout and application state with no backend. Use `mockApi.ts` to simulate short processing delays and load JSON fixtures.
3. Implement syllabus upload, extraction review, course confirmation, additional source cards, and the scripted chat exchange.
4. Verify the complete interaction on a laptop browser and an iPad-sized responsive viewport.
5. Add `research/interaction-methods/webapp/backend` with FastAPI and keep its reusable Python machinery in `research/interaction-methods/src`. Run it locally on a separate port, commonly <http://localhost:8000>, and configure the frontend API base URL through a development environment variable.
6. Replace one mock at a time: health check, provider registry, syllabus fixture response, source registration, then chat. Preserve a mock mode so the UI remains demonstrable without API credentials.
7. Add the Jupyter notebook only after the Python schemas and service boundaries exist. The notebook should import or call those functions rather than contain a second implementation.

The repository should include an `.env.example` listing variable names without values. Actual provider keys stay in an ignored local `.env` file. Setup instructions should specify supported Node.js and Python versions after they are chosen and tested, rather than relying on whichever versions happen to be installed by a contributor.

7.7 Jupyter notebook demonstration

The notebook is the first executable backend experiment, not the primary user interface. It should demonstrate the processing chain with visible intermediate objects:

1. Load `samp_syllabus.pdf` and produce or load the `CourseMetadata` fixture.

2. Confirm the course name and register `samp_notes.pdf`, `samp_homework.pdf`, and one handwritten solution as `SourceArtifact` objects.
3. Construct a `TutorRequest` asking for feedback on the student’s proof.
4. Run the mock orchestrator first, then replace it with one selected model/provider.
5. Display retrieved evidence, interpreted equations, inferred intent, the final `TutorResponse`, citations, and the comprehension check.
6. Log latency and estimated or reported usage so the model-and-cost research can be compared with an actual interaction.

The notebook and browser should produce the same response schema. Once that contract works, the FastAPI endpoint can wrap the notebook-tested function and the frontend can replace its mock client with the HTTP client.

7.8 Path toward a manager–worker agent system

The architecture should leave room to promote selected tools into reasoning workers. A later manager agent could understand the request, create a task plan, and delegate open-ended work to agents that independently retrieve evidence, solve a problem, inspect a diagram, critique a derivation, or design a Socratic response. Deterministic operations such as KaTeX rendering, file conversion, stroke capture, and basic transcription should remain services even in that design.

Expansion is justified when evaluation shows that complex requests benefit from independent reasoning, parallel work, or cross-checking. At that stage, the tutoring orchestrator becomes the manager and synthesizes worker outputs into one student-facing answer with source provenance. Until then, a one-orchestrator design is easier to evaluate because failures can be attributed to input processing, retrieval, reasoning, or response generation.

7.9 Office Hours: a restricted future capability

The interface includes an *Office Hours* control to make the product direction concrete, but the initial prototype labels it *Coming Soon* and does not depend on live audio or video. The intended experience simulates a student visiting a professor’s office: the student brings a question or attempted solution, the professor diagnoses the point of confusion, works through one idea at a time, and checks whether the student is following before continuing.

This behavior should also govern ordinary chat. After each substantial explanation or reasoning step, the tutor explicitly checks comprehension with a short question such as “Does that step make sense?”, “What would you substitute next?”, or “Can you explain why that assumption matters?” The follow-up should match the situation: a simple check-in can preserve conversational flow, while a brief retrieval or application question provides stronger evidence of understanding. If the student is unsure, the tutor should rephrase, use a smaller example, or step back to the missing prerequisite instead of continuing the original explanation.

The first Office Hours implementation can be a synchronous text session or push-to-talk conversation with visible transcripts. Video input is intentionally deferred; it is not necessary to reproduce the core educational behavior of office hours. Synchronous voice can be enabled after the system demonstrates reliable source grounding, intent handling, interruption behavior, transcript quality, latency, cost controls, and session recovery. The eventual mode reuses the same Sources tab, provider configuration, artifact store, and tutoring policies rather than becoming a separate application.

In the future public-facing version, clicking `emphOffice Hours` should enter a dedicated live session

and activate a digital professor avatar. The avatar should synchronize spoken responses with facial movement and simple instructional gestures while the student communicates through real-time text, audio, and optional video. A shared work surface can display equations, code, uploaded pages, and annotations beside the avatar. The system should retain the current course sources and conversation context, provide a live transcript, support interruption and turn-taking, and continue checking whether the student follows. Camera use should be optional and permission-based; students should still be able to complete an Office Hours session using text, voice, and shared artifacts alone. This concept requires later evaluation of streaming latency, speech and avatar synchronization, accessibility, privacy, session recording, and recovery from dropped connections.

8 Prototype priorities and feasibility tests

1. **First demonstrable slice:** the three-tab web interface, syllabus upload and confirmed course-name extraction, open-ended text dialogue, source management, provider configuration, one selected PDF page or photo, rendered and editable equations, and source-linked feedback. Demonstrate a full attempt–hint–revision loop in statistics and an engineering topic.
2. **Small extensions:** push-to-talk, optional TTS, and an explicit-submit stylus canvas. Each should fall back to the existing text/image path if recognition fails.
3. **Subsequent experiments:** corpus retrieval, long recordings, improved stroke recognition, and bounded symbolic/numerical checks. Add only the services needed by observed failures.
4. **Later research:** enable *Office Hours* first as synchronous text or interruptible voice without video; evaluate manager–worker reasoning for complex requests; then consider optional visual input, native PencilKit recognition, and synchronized devices. Compare each addition’s learning and usability benefit against its complexity.

Evaluation proposal, not reported results. Assemble 30–50 representative prompts spanning statistics and engineering, with typed references and matched audio, photo, or ink variants. Include unclear inputs and questions not answered by the supplied material. Use consenting pilot participants and preserve only the data needed for the study.

- **Recognition:** measure transcript word error rate, but separately score critical words, signs, subscripts, and full-expression correctness. Record how often users must correct input and how long correction takes.
- **Grounding and tutoring:** have a knowledgeable reviewer assess correctness, assumption checking, source support, hint usefulness, and whether feedback addresses the student’s actual step. Score perception errors separately from reasoning errors.
- **Responsiveness and reliability:** report median and 95th-percentile time from submission to useful feedback, time to first audio, upload failures, and recovery after network interruption. An initial target might be useful text feedback within five seconds for a short request; this is an engineering target to validate, not a provider guarantee.
- **Cost:** log actual token, audio-duration, OCR-page, storage, and hosting usage per completed task. Compare total cost per successful tutoring interaction, including retries and corrections, rather than isolated model-call prices.
- **Learning and usability:** compare text-only with added modalities on matched tasks, counterbalance order, and collect completion time, correction burden, preference, and a short transfer question. Report uncertainty and participant-level variation; a small convenience pilot supports feasibility findings, not broad learning-effect claims.

An integration is worth keeping when it reduces student effort or improves demonstrated feedback quality. Log model/version and input conditions so results remain interpretable as services change. Make

recording explicit, retain text alternatives, and define deletion and course-access behavior before using real student work. Full autonomous grading, institutional integration, and replacing every professor responsibility remain outside this initial interaction demonstration.

Part III

Related Notes

9 Training and tuning for mathematical capability

The project may eventually adapt an open-weight language or vision-language model using existing mathematical datasets. This could improve mathematical notation, intermediate reasoning, tool use, or recognition of visually presented problems. It should follow a measured baseline using prompting, course-document retrieval, and external math tools. Fine-tuning is an optimization step, not a substitute for grounding, symbolic verification, or tutoring-policy design.

There are also two separate targets. *Mathematical problem solving* concerns arriving at a correct solution. *Mathematical tutoring* concerns diagnosing the student’s current understanding, selecting an appropriate hint, asking a useful question, checking comprehension, and avoiding unnecessary answer disclosure. Most math datasets train or evaluate the first target. A Digital Professor will need additional tutoring dialogues and preference data for the second.

9.1 Candidate datasets and intended use

Dataset	Contents	Potential project use	Important note
NuminaMath [32]	A collection of competition-style math problems and generated solution traces, including chain-of-thought and tool-integrated-reasoning variants.	Supervised fine-tuning for written mathematical reasoning or tool-use formats.	Filter incorrect, duplicated, overly long, and style-inappropriate solutions; verify the specific release and license.
MATH-Vision (MATH-V) [33]	3,040 visual competition problems spanning multiple mathematical disciplines and difficulty levels.	Multimodal evaluation of diagrams, geometry, plots, and other visual context.	Prefer it as a held-out benchmark. Training on evaluation examples would invalidate comparison results.
MATH [34]	Competition mathematics problems with step-by-step solutions in natural language and LaTeX.	Train on the official training split; evaluate mathematical reasoning by subject and difficulty.	Preserve test separation and check for overlap with derivative datasets.

Dataset	Contents	Potential project use	Important note
OpenMathInstruct-2 [35]	Large-scale synthetic math instruction data with problem-solution pairs and released training resources.	Supervised fine-tuning of an open-weight base or instruction model.	Scale does not guarantee correctness; sample, verify, deduplicate, and filter before training.
MetaMathQA [36]	Augmented mathematical questions and responses derived from common reasoning datasets.	Lower-cost experiment for mathematical instruction tuning and question reformulation.	Synthetic augmentation can repeat source patterns and model-specific errors.
GSM8K [37]	Approximately 8,500 linguistically diverse grade-school word problems.	A small, interpretable baseline for arithmetic word-problem reasoning and verifier experiments.	Narrower and easier than the intended university statistics and engineering scope.
DeepMind Mathematics Dataset [38]	Generated school-level questions across arithmetic, algebra, calculus, measurement, probability, and related categories.	Controlled curriculum experiments, data generation, and stress tests with known answers.	Template-generated examples may not resemble authentic student misconceptions or course language.
MathV360K / Math-LLaVA [39]	Multimodal mathematical question-answer pairs collected and synthesized from multiple visual-math sources.	Candidate visual instruction-tuning data for an open vision-language model.	Audit licenses and provenance across constituent sources and hold out independent visual benchmarks.
MathVista [40]	A 6,141-example benchmark combining mathematical and visual reasoning tasks.	Evaluation across visual, scientific, statistical, algebraic, and geometric reasoning.	The official repository describes it as a test set and prohibits using it as training data.

9.2 Recommended tuning sequence

1. **Establish an untuned baseline.** Evaluate a hosted or open-weight model on representative statistics and engineering questions, handwritten or visual inputs, course-grounded questions, and tutoring conversations. Record mathematical correctness and pedagogical quality separately.
2. **Build clean evaluation sets first.** Reserve MATH-Vision, MathVista, official test splits, and a private project set before assembling training data. Deduplicate semantically as well as by exact text because many datasets derive from MATH, GSM8K, competitions, or one another.
3. **Use supervised fine-tuning selectively.** Begin with a small, filtered mixture such as NuminaMath, OpenMathInstruct-2, MATH training data, or MetaMathQA. Normalize answer formats and retain

provenance. Compare the tuned checkpoint against the same frozen baseline.

4. **Add verifier or tool-use behavior.** Train examples in which the model calls a constrained symbolic or numerical tool, checks units and assumptions, and reports when a result cannot be verified. Correct tool traces matter more than verbose reasoning text.
5. **Tune tutoring behavior separately.** Create or curate dialogues containing a student attempt, diagnosed misconception, next instructional move, comprehension check, and response to uncertainty. Use expert review or preference optimization so the model learns to teach rather than simply reveal solutions.
6. **Consider multimodal tuning only after text experiments.** Visual fine-tuning requires an appropriate vision-language base model, more memory, careful image preprocessing, and independent visual evaluation. MATH-Vision and MathVista should remain benchmarks; a training-oriented resource such as MathV360K is a more plausible starting point after its provenance is reviewed.

9.3 Practical risks and decision criteria

Dataset licenses, terms, image rights, and model licenses must be reviewed for the exact versions used. Dataset cards can change, and a permissive code license does not automatically grant unrestricted rights to every underlying problem or image. Store the dataset version, source URL, license information, transformations, and split assignment in a data manifest.

Generated solution traces may be fluent but wrong. Before tuning, validate final answers where possible with symbolic or numerical tools and manually inspect samples across topics. Remove answer-only examples if the desired behavior is explanatory, and avoid training the model to expose long solutions when the tutoring policy calls for hints and comprehension checks.

Proceed with tuning only if it improves a defined failure mode enough to justify training and serving costs. Useful measures include exact or symbolic answer correctness, diagram-dependent accuracy, unit and assumption handling, source grounding, misconception diagnosis, hint quality, comprehension checking, latency, and cost per completed tutoring interaction. A stronger MATH score alone would not establish a better Digital Professor.

References

- [1] OpenAI. *Text generation*. <https://developers.openai.com/api/docs/guides/text>.
- [2] OpenAI. *Realtime and audio*. <https://developers.openai.com/api/docs/guides/realtime>.
- [3] Google. *Gemini Live API overview*. <https://ai.google.dev/gemini-api/docs/live-api>.
- [4] LiveKit. *Agents documentation: Introduction*. <https://docs.livekit.io/agents/>.
- [5] Mozilla contributors. *MediaRecorder*. <https://developer.mozilla.org/en-US/docs/Web/API/MediaRecorder>.
- [6] OpenAI. *File transcription*. <https://developers.openai.com/api/docs/guides/speech-to-text>.
- [7] OpenAI. *Text to speech*. <https://developers.openai.com/api/docs/guides/text-to-speech>.
- [8] SYSTRAN and contributors. *faster-whisper*. <https://github.com/SYSTRAN/faster-whisper>.
- [9] Mozilla contributors. *Web Speech API*. https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API.
- [10] Mozilla. *PDF.js*. <https://mozilla.github.io/pdf.js/>.

- [11] OpenAI. *File inputs*. <https://developers.openai.com/api/docs/guides/file-inputs>.
- [12] OpenAI. *File search*. <https://developers.openai.com/api/docs/guides/tools-file-search>.
- [13] Docling contributors. *Docling: document processing for generative AI*. <https://github.com/docling-project/docling>.
- [14] Mozilla contributors. *Pointer events*. https://developer.mozilla.org/en-US/docs/Web/API/Pointer_events.
- [15] MyScript. *iinkTS: Get started*, Interactive Ink 4.1 documentation. <https://developer.myscript.com/doc/interactive-ink/4.1/web/iinkts/get-started/>.
- [16] Mathpix. *Mathpix OCR API: Introduction*. <https://docs.mathpix.com/>.
- [17] MathLive. *Mathfield: Introduction*. <https://mathlive.io/mathfield/>.
- [18] KaTeX. *API*. <https://katex.org/docs/api>.
- [19] SymPy contributors. *Parsing*. <https://docs.sympy.org/latest/modules/parsing.html>.
- [20] Mozilla contributors. *MediaDevices: getUserMedia() method*. <https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia>.
- [21] OpenAI. *Images and vision*. <https://developers.openai.com/api/docs/guides/images-vision>.
- [22] Apple. *PencilKit*. <https://developer.apple.com/documentation/pencilkit>.
- [23] Google. *Video understanding*. <https://ai.google.dev/gemini-api/docs/video-understanding>.
- [24] Graesser, A. C., VanLehn, K., Rose, C. P., Jordan, P. W., and Harter, D. (2001). *Intelligent Tutoring Systems with Conversational Dialogue*. AI Magazine, 22(4), 39. <https://doi.org/10.1609/aimag.v22i4.1591>.
- [25] Anthony, L., Yang, J., and Koedinger, K. R. (2012). *A paradigm for handwriting-based intelligent tutors*. International Journal of Human–Computer Studies, 70(11), 866–887. <https://doi.org/10.1016/j.ijhcs.2012.04.003>.
- [26] de Moraes, F., and Jaques, P. A. (2022). *Does handwriting impact learning on math tutoring systems?* Informatics in Education, 21(1), 55–90. <https://doi.org/10.15388/infedu.2022.03>.
- [27] Rodrigues, L., Pereira, F. D., Marinho, M., Macario, V., Bittencourt, I. I., Isotani, S., Dermeval, D., and Mello, R. (2024). *Mathematics intelligent tutoring systems with handwritten input: A scoping review*. Education and Information Technologies, 29, 11183–11209. <https://doi.org/10.1007/s10639-023-12145-3>.
- [28] Moreno, R., and Mayer, R. E. (2002). *Verbal redundancy in multimedia learning: When reading helps listening*. Journal of Educational Psychology, 94(1), 156–163. <https://doi.org/10.1037/0022-0663.94.1.156>.
- [29] Maurya, K. K., et al. (2025). *Unifying AI Tutor Evaluation: An Evaluation Taxonomy for Pedagogical Ability Assessment of LLM-Powered AI Tutors*. Proceedings of NAACL 2025. <https://aclanthology.org/2025.naacl-long.57/>.
- [30] Liu, M., Wang, L., and Zhang, W. (2025). *Is Your Multimodal Large Language Model a Good Science Tutor?* <https://arxiv.org/abs/2505.06418>.
- [31] Wang, R., et al. (2024). *Stepwise Verification and Remediation of Student Reasoning Errors with Large Language Model Tutors*. <https://arxiv.org/abs/2407.09136>.
- [32] AI-MO. *NuminaMath collection*. <https://huggingface.co/collections/AI-MO/numinamath>.
- [33] Wang, K., Pan, J., Shi, W., Lu, Z., Ren, H., Zhou, A., Zhan, M., and Li, H. (2024). *Measuring Multimodal Mathematical Reasoning with the MATH-Vision Dataset*. NeurIPS 2024 Datasets and Benchmarks Track. <https://github.com/mathllm/MATH-V>.

- [34] Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. (2021). *Measuring Mathematical Problem Solving With the MATH Dataset*. NeurIPS 2021. <https://github.com/hendrycks/math>.
- [35] Toshniwal, S., Du, W., Moshkov, I., Kisacanin, B., Ayrapetyan, A., and Gitman, I. (2024). *OpenMathInstruct-2: Accelerating AI for Math with Massive Open-Source Instruction Data*. <https://huggingface.co/datasets/nvidia/OpenMathInstruct-2>.
- [36] Yu, L., et al. (2023). *MetaMath: Bootstrap Your Own Mathematical Questions for Large Language Models*. <https://github.com/meta-math/MetaMath>.
- [37] Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. (2021). *Training Verifiers to Solve Math Word Problems*. <https://github.com/openai/grade-school-math>.
- [38] Saxton, D., Grefenstette, E., Hill, F., and Kohli, P. (2019). *Analysing Mathematical Reasoning Abilities of Neural Models*. https://github.com/google-deepmind/mathematics_dataset.
- [39] Shi, W., et al. (2024). *Math-LLaVA: Bootstrapping Mathematical Reasoning for Multimodal Large Language Models*. <https://github.com/HZQ950419/Math-LLaVA>.
- [40] Lu, P., et al. (2024). *MathVista: Evaluating Mathematical Reasoning of Foundation Models in Visual Contexts*. <https://github.com/lupantech/MathVista>.
- [41] Meta Open Source. *React: Learn*. <https://react.dev/learn>.
- [42] Vite contributors. *Vite: Getting Started*. <https://vite.dev/guide/>.
- [43] FastAPI contributors. *FastAPI Tutorial*. <https://fastapi.tiangolo.com/tutorial/>.